ProofPay

Blockchain-Based Payment Escrow for Freelancers

Project type: Real-world FinTech + Blockchain portfolio project

Primary goal: Build a production-style escrow system where client funds are locked in a smart contract and released according to milestone-based conditions.

1. Real-World Problem

Freelancers and international clients often face a trust problem: the freelancer wants payment security before starting, while the client wants proof of delivery before releasing payment. Traditional escrow introduces a centralized intermediary. ProofPay explores a blockchain-based alternative in which the payment rules are enforced by a smart contract.

2. Core User Flow

Step	Action
1	Client creates an escrow project.
2	Client deposits test funds into the smart contract.
3	Client defines milestones and amounts.
4	Freelancer submits work for a milestone.
5	Client approves the milestone.
6	Smart contract releases that milestone's funds.
7	If there is a disagreement, either party can open a dispute.

3. System Architecture

The key architectural principle is that the backend should not control escrowed funds. The smart contract owns the on-chain escrow state, while the NestJS backend handles application data, indexing, APIs, notifications, and off-chain metadata.

```
Next.js Frontend → NestJS API → PostgreSQL / Redis
■ Blockchain Smart Contract
Blockchain → Event Indexer → PostgreSQL → Frontend
```

4. Technology Stack

Layer	Technology
Frontend	Next.js / React
Backend	NestJS / TypeScript
Database	PostgreSQL
Caching / temporary state	Redis
Blockchain	Ethereum-compatible local chain or Sepolia testnet

Smart contracts	Solidity + Hardhat
Blockchain integration	ethers.js or viem
Infrastructure	Docker + CI/CD
Testing	Jest + Solidity contract tests

5. Smart Contract MVP

- **createEscrow()** — creates a project and establishes participants.
- **deposit()** — locks test funds in the contract.
- **addMilestone()** — defines milestone amount and state.
- **submitMilestone()** — records that work was submitted.
- **approveMilestone()** — approves a completed milestone.
- **releasePayment()** — transfers the milestone amount according to contract rules.
- **openDispute()** — moves a milestone into dispute state.
- **refund()** — supports an explicitly defined refund path.

6. Proof of Delivery

Do not put large project files directly on-chain. Instead, calculate a cryptographic hash of a submitted artifact and store the hash or a reference to it. A later download can be hashed again and compared against the original value.

```
File → SHA-256 → content hash → recorded with milestone → later verification
```

This demonstrates a practical connection between blockchain immutability and digital-asset verification.

7. Dispute System

- Client can submit evidence.
- Freelancer can submit evidence.
- Milestone history and timestamps are preserved.
- A designated arbitrator can resolve the dispute in the MVP.
- The contract enforces the resulting state transition.

8. Backend Modules

- Auth / wallet identity
- Users
- Escrows
- Milestones
- Blockchain transactions
- Blockchain event indexer
- Proof-of-delivery metadata
- Disputes
- Notifications
- Audit logs

9. Suggested API

- POST /escrows
- GET /escrows/:id

- POST /escrows/:id/milestones
- POST /milestones/:id/submit
- POST /milestones/:id/approve
- POST /milestones/:id/dispute
- GET /escrows/:id/transactions
- GET /blockchain/events/:escrowId

10. Database Model

Table	Purpose
users	Application profile and wallet identity
escrows	Project, participants, amount, contract address, status
milestones	Amount, status, proof hash, timestamps
blockchain_transactions	Tx hash, block, type, status
blockchain_events	Indexed contract events
disputes	Dispute state, evidence references, resolution
audit_logs	Off-chain application activity

11. Today’s MVP Definition

- Create a local Ethereum-compatible blockchain or use a public testnet.
- Deploy the escrow smart contract.
- Connect a browser wallet.
- Create an escrow.
- Deposit test funds.
- Create at least one milestone.
- Submit and approve the milestone.
- Release the milestone payment from the smart contract.
- Display the resulting transaction hash in the UI.

12. Advanced Roadmap

Phase	Feature
V1	Milestone escrow + wallet + smart contract
V2	Blockchain event indexer + audit trail
V3	Proof-of-delivery hash verification
V4	Dispute and arbitration workflow
V5	Timeouts, cancellation and automated refunds
V6	Multi-chain / cross-chain settlement research
V7	Risk analysis and AI-assisted evidence classification

13. Portfolio / CV Evidence

- Public GitHub repository with clean commit history.
- Architecture diagram and technical design document.
- Automated unit, integration, and smart-contract tests.

- Benchmark or load-test results where meaningful.
- Live frontend and testnet contract address.
- Short demo video showing the complete escrow lifecycle.
- README explaining design decisions, trade-offs, security considerations, and failure cases.

14. Example CV Bullet

ProofPay — Blockchain Escrow Platform
Designed and implemented a milestone-based blockchain escrow platform using Solidity, NestJS, PostgreSQL and Redis; enforced payment state transitions through smart contracts, built a blockchain event indexer, and added cryptographic proof-of-delivery verification and dispute workflows.

15. Important Development Rule

Use only local blockchain networks or testnets during development. Do not handle real customer funds or present the MVP as a regulated financial service. Treat production custody, payments, KYC/AML, consumer protection, and jurisdiction-specific regulation as separate engineering and legal requirements.